tf.where Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/where

> *Returns the indices of non-zero elements, or multiplexes x and y.*

Function Signatures

```
tf.where( condition, x=None, y=None, name=None )
```

Code Examples

```
tf.where(
condition, x=None, y=None, name=None
)
```

```
tf.where(
condition, x=None, y=None, name=None
)
```

```
tf.where([True, False, False, True]).numpy()
array([[0],
[3]])
```

```
tf.where([True, False, False, True]).numpy()
```

```
array([[0],
```

```
tf.where([[1, 0, 0], [1, 0, 1]]).numpy()
array([[0, 0],
[1, 0],
[1, 2]])
```

```
tf.where([[1, 0, 0], [1, 0, 1]]).numpy()
```

Documentation

Returns the indices of non-zero elements, or multiplexes x and y.

View aliases

Compat aliases for migration

See
Migration guide for
more details.

tf.compat.v1.where_v2

Used in the notebooks

• Extension types

- Better performance with tf.function
- Unicode strings
- Transfer learning and fine-tuning
- Integrated gradients
- Substantial Undocumented Infection Facilitates the Rapid Dissemination of Novel Coronavirus (SARS-CoV2)
- TFP Release Notes notebook (0.11.0)
- Bayesian Switchpoint Analysis

This operation has two modes:

- Return the indices of non-zero elements - When only

condition is provided the result is an int64 tensor where each row is
the index of a non-zero element of condition. The result's shape
is [tf.math.count_nonzero(condition), tf.rank(condition)].

- Multiplex x and y - When both x and y are provided the

result has the shape of x, y, and condition broadcast together. The result is taken from x where condition is non-zero or y where condition is zero.

1. Return the indices of non-zero elements

If x and y are not provided (both are None):

tf.where will return the indices of condition that are non-zero, in the form of a 2-D tensor with shape [n, d], where n is the number of non-zero elements in condition (tf.count_nonzero(condition)), and d is the number of axes of condition (tf.rank(condition)).

Indices are output in row-major order. The condition can have a dtype of tf.bool, or any numeric dtype.

Here condition is a 1-axis bool tensor with 2 True values. The result has a shape of [2,1]

Here condition is a 2-axis integer tensor, with 3 non-zero values. The result has a shape of [3, 2].

Here condition is a 3-axis float tensor, with 5 non-zero values. The output shape is [5, 3].

These indices are the same that tf.sparse.SparseTensor would use to represent the condition tensor:

A complex number is considered non-zero if either the real or imaginary component is non-zero:

2. Multiplex x and y

If x and y are also provided (both have non-None values) the condition tensor acts as a mask that chooses whether the corresponding element / row in the output should be taken from x (if the element in condition is True) or y (if it is False).

The shape of the result is formed by
broadcasting
together the shapes of condition, x, and y.

When all three inputs have the same size, each is handled element-wise.

There are two main rules for broadcasting:

- If a tensor has fewer axes than the others, length-1 axes are added to the

left of the shape.

- Axes with length-1 are streched to match the coresponding axes of the other

tensors.

A length-1 vector is streched to match the other vectors:

A scalar is expanded to match the other arguments:

A scalar condition returns the complete x or y tensor, with
broadcasting applied.

For a non-trivial example of broadcasting, here condition has a shape of [3], x has a shape of [3,3], and y has a shape of [3,1].

Broadcasting first expands the shape of condition to [1,3]. The final broadcast shape is [3,3]. condition will select columns from x and y. Since y only has one column, all columns from y will be identical.

Note that if the gradient of either branch of the tf.where generates a NaN, then the gradient of the entire tf.where will be NaN. This is because the gradient calculation for tf.where combines the two branches, for performance reasons.

A workaround is to use an inner tf.where to ensure the function has no asymptote, and to avoid computing a value whose gradient is NaN by replacing dangerous inputs with safe inputs.

Instead of this,

Although, the 1. / x values are never used, its gradient is a NaN when x = 0. Instead, we should guard that with another tf.where

See also:

• tf.sparse - The indices returned by the first form of tf.where can be useful in tf.sparse.SparseTensor objects.

- tf.gather_nd, tf.scatter_nd, and related ops - Given the

list of indices returned from tf.where the scatter and gather family of ops can be used fetch values or insert values at those indices.

- tf.strings.length - tf.string is not an allowed dtype for the

condition. Use the string length instead.

Returns

Raises